Amazon S3 i en Fullstack-applikation

> **Kursnivå:** Grundläggande – Intermediär
> **Förutsättningar:** Grundläggande kunskaper i Java/Spring Boot, JavaScript och REST API:er
> **Tidsåtgång:** Ca 3–4 timmar (teori + övningar)

Vad är Amazon S3?

Amazon Simple Storage Service (S3) är en objektlagringstjänst från AWS som låter dig lagra och hämta godtyckliga mängder data när som helst, var som helst på webben.

Till skillnad från ett filsystem på en dator organiserar S3 data som **objekt** i **buckets** (hinkar). Varje objekt består av:

- **Data** – själva filinnehållet (bild, PDF, JSON, video, etc.)
- **Nyckel (Key)** – ett unikt namn inom bucketen (fungerar som en filsökväg, t.ex. `uploads/users/avatar-123.png`)
- **Metadata** – valfri information om objektet (content-type, skapandedatum, etc.)

Varför S3?

Egenskap	Beskrivning
Skalbarhet	Lagrar obegränsat med data
Tillgänglighet	99.999999999% (11 nior) hållbarhet
Kostnad	Betala bara för det du använder
Integration	Sömlös integration med övriga AWS-tjänster
Säkerhet	Finkornad åtkomstkontroll via IAM och bucket policies

---

Grundläggande Begrepp

Bucket

En bucket är en behållare för objekt. Varje bucket har ett **globalt unikt namn** över hela AWS. Du väljer också i vilken **region** (geografisk plats) bucketen ska skapas – välj nära dina användare för lägst latens.

```
s3://min-applikation-uploads/
```

Object Key

Nyckeln är objektets "sökväg" i bucketen. S3 är egentligen platt (ingen riktig mappstruktur), men `/` i nyckelnamn ger en mappliknande upplevelse i AWS Console.

```
uploads/images/profil-bilder/user-42.jpg
uploads/documents/2024/rapport.pdf
```

Åtkomstkontroll

S3 har flera lager av åtkomstkontroll:

- **Block Public Access** – standardinställning som blockerar all offentlig åtkomst
- **Bucket Policy** – JSON-baserade regler för vilka som får göra vad
- **IAM Roles/Policies** – styr vilka AWS-resurser (EC2, Lambda, etc.) som får använda bucketen
- **Pre-signed URLs** – tillfälliga, signerade URL:er för tidsbegränsad åtkomst

---

Arkitektur: Vår Fullstack-applikation

I den här lektionen arbetar vi med en arkitektur som ser ut så här:

```
┌──────────────────┐
│                  │
│   Frontend       │
│   (React/Vue)    │
│                  │
└──────────────────┘
                    HTTP
                    ──────────►
                                ┌──────────────────────────┐
                                │                          │
                                │   HTTP API (API Gateway) │
                                │                          │
                                └──────────────────────────┘
                                          │       │
                                          ▼       ▼
                                ┌──────────────┐  ┌──────────────────┐
                                │              │  │                  │
                                │  Lambda      │  │  Spring Boot     │
                                │  Funktioner  │  │  Backend (EC2)   │
                                │              │  │                  │
                                └──────────────┘  └──────────────────┘
                                          │       │
                                          └───────┴───────┐
                                                          │
                                                          ▼
                                                  ┌──────────────┐
                                                  │              │
                                                  │  Amazon S3   │
                                                  │   Bucket     │
                                                  │              │
                                                  └──────────────┘
```

Ansvarsområden

Frontend (React/Vue)

- Tillåter användaren att ladda upp filer
- Hämtar och visar filer/bilder från S3
- Kommunicerar med backend via HTTP API

HTTP API (API Gateway)

- Tar emot HTTP-anrop från frontend
- Routar anrop till rätt Lambda-funktion

Lambda-funktioner

- Hanterar lättviktiga operationer: generera pre-signed URLs, validering
- Svarar snabbt utan att behöva hålla en server igång

Spring Boot Backend (EC2)

- Hanterar komplex affärslogik
- Processning av uppladdade filer (t.ex. bildresizing, dokumentanalys)
- Kommunicerar direkt med S3 via AWS SDK

S3 Bucket

- Lagrar alla filer/objekt
- Tillgänglig från både Lambda och EC2

S3 och Lambda: Pre-signed URLs

Ett vanligt mönster är att låta **frontend ladda upp direkt till S3** utan att filen behöver passera genom din backend. Detta kallas **pre-signed URL upload**.

Flöde

```
1. Frontend ──GET /upload-url?filename=bild.jpg──▶ Lambda
2. Lambda  ──genererar pre-signed PUT URL──────▶ S3
3. Lambda  ◀──returnerar URL (giltig 15 min)──────
4. Frontend ◀──{ uploadUrl: "https://s3.aws..." }──
5. Frontend ──PUT bild.jpg direkt till S3──────────▶ S3
```

Lambda-funktion: Generera Pre-signed URL (Node.js)

```
// generate-upload-url/index.mjs
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";
import { randomUUID } from "crypto";

const s3 = new S3Client({ region: process.env.AWS_REGION });
const BUCKET_NAME = process.env.S3_BUCKET_NAME;

export const handler = async (event) => {
  const { filename, contentType } = JSON.parse(event.body);

  // Säker nyckel — använd aldrig användarens filnamn direkt!
  const key = `uploads/${randomUUID()}-${filename}`;
```

```
  const command = new PutObjectCommand({
    Bucket: BUCKET_NAME,
    Key: key,
    ContentType: contentType,
  });

  // URL giltig i 15 minuter
  const uploadUrl = await getSignedUrl(s3, command, { expiresIn: 900 });

  return {
    statusCode: 200,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ uploadUrl, key }),
  };
};
```

Lambda-funktion: Hämta Pre-signed URL för nedladdning (Node.js)

```
// generate-download-url/index.mjs
import { S3Client, GetObjectCommand } from "@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";

const s3 = new S3Client({ region: process.env.AWS_REGION });

export const handler = async (event) => {
  const { key } = event.queryStringParameters;

  const command = new GetObjectCommand({
    Bucket: process.env.S3_BUCKET_NAME,
    Key: key,
  });

  const downloadUrl = await getSignedUrl(s3, command, { expiresIn: 3600 });

  return {
    statusCode: 200,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ downloadUrl }),
  };
};
```

S3 och Spring Boot: AWS SDK för Java

Din Spring Boot-backend på EC2 interagerar med S3 via **AWS SDK for Java v2**.

Beroenden (pom.xml)

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>bom</artifactId>
      <version>2.25.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId>
  </dependency>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3-transfer-manager</artifactId>
  </dependency>
</dependencies>
```

S3-konfiguration (Spring Boot)

```
// config/S3Config.java
@Configuration
public class S3Config {

    @Value("${aws.region}")
    private String region;

    @Bean
    public S3Client s3Client() {
        return S3Client.builder()
                .region(Region.of(region))
                // På EC2 med IAM Role behövs ingen explicit autentisering!
                // SDK:n hämtar credentials automatiskt från EC2 Instance
Metadata
                .credentialsProvider(DefaultCredentialsProvider.create())
                .build();
    }
}
```

S3-tjänst (Spring Boot)

```
// service/S3Service.java
@Service
```

```
public class S3Service {

    private final S3Client s3Client;

    @Value("${aws.s3.bucket-name}")
    private String bucketName;

    public S3Service(S3Client s3Client) {
        this.s3Client = s3Client;
    }

    /**
     * Ladda upp en fil till S3
     */
    public String uploadFile(String key, byte[] data, String contentType) {
        PutObjectRequest request = PutObjectRequest.builder()
                .bucket(bucketName)
                .key(key)
                .contentType(contentType)
                .build();

        s3Client.putObject(request, RequestBody.fromBytes(data));
        return key;
    }

    /**
     * Ladda ned en fil från S3
     */
    public byte[] downloadFile(String key) {
        GetObjectRequest request = GetObjectRequest.builder()
                .bucket(bucketName)
                .key(key)
                .build();

        ResponseBytes<GetObjectResponse> objectBytes =
s3Client.getObjectAsBytes(request);
        return objectBytes.asByteArray();
    }

    /**
     * Ta bort ett objekt från S3
     */
    public void deleteFile(String key) {
        DeleteObjectRequest request = DeleteObjectRequest.builder()
                .bucket(bucketName)
                .key(key)
                .build();

        s3Client.deleteObject(request);
    }

    /**
     * Lista alla objekt med ett visst prefix (t.ex. alla filer för en
användare)
```

```
     */
    public List<String> listFiles(String prefix) {
        ListObjectsV2Request request = ListObjectsV2Request.builder()
                .bucket(bucketName)
                .prefix(prefix)
                .build();

        ListObjectsV2Response response = s3Client.listObjectsV2(request);
        return response.contents().stream()
                .map(S3Object::key)
                .collect(Collectors.toList());
    }
}
```

REST-kontroller (Spring Boot)

```
// controller/FileController.java
@RestController
@RequestMapping("/api/files")
public class FileController {

    private final S3Service s3Service;

    public FileController(S3Service s3Service) {
        this.s3Service = s3Service;
    }

    @PostMapping("/upload")
    public ResponseEntity<Map<String, String>> uploadFile(
            @RequestParam("file") MultipartFile file) throws IOException {

        String key = "processed/" + UUID.randomUUID() + "-" +
file.getOriginalFilename();
        s3Service.uploadFile(key, file.getBytes(), file.getContentType());

        return ResponseEntity.ok(Map.of("key", key, "status", "uploaded"));
    }

    @GetMapping("/download/{key}")
    public ResponseEntity<byte[]> downloadFile(@PathVariable String key) {
        byte[] data = s3Service.downloadFile(key);

        return ResponseEntity.ok()
                .header(HttpHeaders.CONTENT_DISPOSITION, "attachment;
filename=\"file\"")
                .body(data);
    }

    @DeleteMapping("/{key}")
    public ResponseEntity<Void> deleteFile(@PathVariable String key) {
        s3Service.deleteFile(key);
```

```
        return ResponseEntity.noContent().build();
    }
}
```

application.properties

```
aws.region=eu-north-1
aws.s3.bucket-name=min-applikation-uploads
```

Frontend: Uppladdning med Pre-signed URL

```
// services/fileService.js (Vue/React – fungerar i båda)

const API_BASE = "https://din-api-gateway-url.amazonaws.com";

/**
 * Steg 1: Hämta en pre-signed URL från Lambda
 * Steg 2: Ladda upp filen direkt till S3
 */
export async function uploadFile(file) {
  // 1. Be Lambda om en uppladdnings-URL
  const urlResponse = await fetch(`${API_BASE}/upload-url`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      filename: file.name,
      contentType: file.type,
    }),
  });

  const { uploadUrl, key } = await urlResponse.json();

  // 2. Ladda upp direkt till S3 med den pre-signerade URL:en
  const uploadResponse = await fetch(uploadUrl, {
    method: "PUT",
    headers: { "Content-Type": file.type },
    body: file, // Skicka filen direkt – ingen base64-kodning behövs
  });

  if (!uploadResponse.ok) {
    throw new Error("Uppladdning misslyckades");
  }

  return key; // Spara nyckeln i din databas för framtida hämtning
}

/**
 * Hämta en nedladdnings-URL för att visa/ladda ned en fil
```

```
 */
export async function getDownloadUrl(key) {
  const response = await fetch(
    `${API_BASE}/download-url?key=${encodeURIComponent(key)}`
  );
  const { downloadUrl } = await response.json();
  return downloadUrl;
}
```

React-komponent: Filuppladdning

```
// components/FileUpload.jsx
import { useState } from "react";
import { uploadFile } from "../services/fileService";

export default function FileUpload() {
  const [status, setStatus] = useState("idle");
  const [uploadedKey, setUploadedKey] = useState(null);

  const handleFileChange = async (e) => {
    const file = e.target.files[0];
    if (!file) return;

    setStatus("uploading");
    try {
      const key = await uploadFile(file);
      setUploadedKey(key);
      setStatus("success");
    } catch (error) {
      console.error(error);
      setStatus("error");
    }
  };

  return (
    <div>
      <input type="file" onChange={handleFileChange} disabled={status ===
"uploading"} />
      {status === "uploading" && <p>Laddar upp...</p>}
      {status === "success" && <p>✅ Uppladdad! Nyckel: {uploadedKey}</p>}
      {status === "error" && <p>❌ Något gick fel. Försök igen.</p>}
    </div>
  );
}
```

IAM-roller och Säkerhet

EC2 Instance Role

Din EC2-instans bör ha en **IAM Role** med följande policy för att komma åt S3:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::min-applikation-uploads",
        "arn:aws:s3:::min-applikation-uploads/*"
      ]
    }
  ]
}
```

Lambda Execution Role

Lambda-funktionen behöver liknande behörigheter. AWS SDK:n hämtar automatiskt credentials från Lambda-miljön.

Bucket Policy: Blockera all direkt offentlig åtkomst

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyPublicAccess",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::min-applikation-uploads/*",
      "Condition": {
        "StringNotEquals": {
          "aws:PrincipalArn": [
            "arn:aws:iam::DITT-KONTO-ID:role/ec2-spring-boot-role",
            "arn:aws:iam::DITT-KONTO-ID:role/lambda-execution-role"
          ]
        }
      }
    }
  ]
}
```

> ⚠️ **Viktig säkerhetsregel:** Aktivera alltid **Block Public Access** på bucket-nivå om du inte har ett specifikt skäl att göra filer offentliga. Använd pre-signed URLs för att ge tillfällig åtkomst.

CORS-konfiguration för S3

För att frontend ska kunna ladda upp direkt till S3 behöver bucketen en **CORS-konfiguration**:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "PUT", "POST"],
    "AllowedOrigins": ["https://din-frontend-domän.com"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

> Under utveckling kan du temporärt sätta `"AllowedOrigins": ["*"]`, men byt ut detta mot din riktiga domän i produktion.

Övningar

Övning 1 – Skapa och konfigurera en S3-bucket (30 min)

Mål: Bekanta dig med AWS Console och grundläggande S3-konfiguration.

1. Logga in på AWS Console och navigera till S3.
2. Skapa en ny bucket med ett unikt namn (t.ex. `[ditt-namn]-lab-uploads`).
   - Välj region `eu-north-1` (Stockholm)
   - Lämna "Block all public access" aktiverat
3. Ladda upp en testfil (t.ex. en bild) manuellt via konsolen.
4. Försök öppna filen via dess URL – vad händer och varför?
5. Skapa en pre-signed URL via konsolen (Actions → Share with a presigned URL) och testa att öppna den.

Reflektionsfrågor:

- Varför fungerade inte den vanliga URL:en?
- Hur länge är din pre-signed URL giltig?

Övning 2 – Lambda-funktion för pre-signed URL (45 min)

Mål: Skapa en Lambda-funktion som genererar pre-signed upload-URL:er.

1. Skapa en ny Lambda-funktion med Node.js 20.x runtime.
2. Ge Lambda-funktionen en IAM execution role med `s3:PutObject`-behörighet till din bucket.
3. Implementera koden från avsnittet "Lambda-funktion: Generera Pre-signed URL" ovan.

4. Lägg till miljövariabeln `S3_BUCKET_NAME` i Lambda-konfigurationen.
5. Testa funktionen med följande test-event:

```
{
  "body": "{\"filename\": \"test.jpg\", \"contentType\": \"image/jpeg\"}"
}
```

6. Kopiera `uploadUrl` från svaret och testa att ladda upp en fil med `curl`:

```
curl -X PUT \
  -H "Content-Type: image/jpeg" \
  --data-binary "@din-bild.jpg" \
  "DIN_PRESIGNED_URL_HÄR"
```

7. Verifiera i S3-konsolen att filen dök upp i bucketen.

Bonusutmaning: Lägg till validering så att bara `image/jpeg` och `image/png` accepteras.

Övning 3 – Spring Boot-integration (60 min)

Mål: Integrera S3 i ett Spring Boot-projekt.

1. Skapa ett nytt Spring Boot-projekt med Spring Web och AWS SDK-beroendet.
2. Kopiera `S3Config.java` och `S3Service.java` från lektionen.
3. Konfigurera `application.properties` med din bucket-information.
4. Skapa en REST-endpoint `/api/files/upload` som tar emot en fil och laddar upp den till S3.
5. Skapa en endpoint `/api/files/{key}` som laddar ned och returnerar en fil från S3.

Testa lokalt med AWS CLI-profil:
Se till att du har en `~/.aws/credentials`-fil med ditt AWS-konto, eller kör applikationen på en EC2 med korrekt IAM Role.

```
# Testa uppladdning
curl -X POST http://localhost:8080/api/files/upload \
  -F "file=@din-bild.jpg"

# Testa nedladdning (ersätt KEY med returvärdet ovan)
curl http://localhost:8080/api/files/download/KEY --output nedladdat.jpg
```

Bonusutmaning: Implementera `listFiles(String prefix)` och exponera den via `/api/files/list?prefix=uploads/`.

Övning 4 – Fullstack-integration (60 min)

Mål: Koppla ihop frontend, Lambda och S3.

1. Skapa ett enkelt React- eller Vue-projekt med Vite:

```
npm create vite@latest min-s3-app -- --template react
cd min-s3-app && npm install
```

2. Kopiera `fileService.js` och `FileUpload.jsx` från lektionen och anpassa API-URL:en.
3. Skapa en HTTP API i API Gateway och lägg till din Lambda-funktion som integration.
4. Konfigurera CORS på din S3-bucket (tillåt `http://localhost:5173` under utveckling).
5. Kör frontend-appen och testa att ladda upp en fil från webbläsaren.
6. Verifiera att filen syns i S3-konsolen med rätt nyckel.

Bonusutmaning:
Lägg till ett bildgalleri i frontend som listar uppladdade bilder med hjälp av nedladdnings-URL:er från din Lambda.

Övning 5 – Säkerhet och kostnadsoptimering (30 min)

Mål: Reflektera kring säkerhet och drift.

Diskutera och implementera (om tid finns) följande:

1. **Filnamnssanering:** Varför är `UUID + originalFilename` bättre än bara `originalFilename` som S3-nyckel?

2. **Filstorleksbegränsning:** Hur kan du begränsa vilken storlek en fil får ha?

   - I Lambda (pre-signed URL): Lägg till `ContentLengthRange` i pre-signed URL-kommandot.
   - I Spring Boot: Använd `spring.servlet.multipart.max-file-size=10MB`.

3. **Livscykelregler:** Gå till din S3-bucket → Management → Lifecycle rules. Skapa en regel som automatiskt tar bort objekt i `temp/`-mappen efter 7 dagar. Varför är detta användbart?

4. **Kostnadskalkyl:** Uppskatta vad det kostar per månad att lagra 100 GB i S3 och serva 1 TB data per månad. Använd AWS Pricing Calculator.

Sammanfattning

I den här lektionen har vi gått igenom:

- Vad S3 är och hur objekt, buckets och nycklar fungerar
- Hur S3 passar in i en fullstack-arkitektur med EC2, Lambda och API Gateway
- Hur Lambda genererar pre-signed URLs för säker direktuppladdning från frontend
- Hur Spring Boot integreras med S3 via AWS SDK for Java
- Säkerhetsaspekter: IAM-roller, bucket policies och CORS

Nästa steg

- Utforska **S3 Event Notifications** – trigga Lambda automatiskt när en fil laddas upp
- Lär dig om **S3 Versioning** för att behålla historik av filer
- Titta på **CloudFront** för att cacha och snabba upp leveransen av S3-filer globalt
- Utforska **S3 Intelligent-Tiering** för automatisk kostnadsoptimering

---

Material framtaget för introduktionskurs i AWS och fullstack-utveckling.